

Current Architecture and Human-Governed Repair Protocol

A reference architecture for deterministic execution, structured AI diagnosis, approval-gated source mutation, and post-repair verification

T. R. Bentley

Technical Architecture Paper

September 2026

Core claim

AI Debugger Pro treats an AI-generated correction as a proposal, not an action. Deterministic execution establishes evidence; the model produces a structured diagnosis; the user authorizes mutation; the system reruns the result before recording a repair as verified.

Reference implementation: Tybent18/ai-debugger-pro
Supported execution languages: Python, C, C++, Java
Default execution backend: restricted Docker

Abstract

AI-assisted debugging systems face a control problem that is separate from model capability: how should probabilistic recommendations interact with executable source code? AI Debugger Pro addresses this by separating evidence collection, model interpretation, repair proposal, human authorization, mutation, and verification into explicit architectural stages. The current prototype supports Python, C, C++, and Java; defaults to restricted Docker execution; captures compiler/runtime evidence and bounded Python exception state; requests structured AI diagnoses with a deterministic offline fallback; previews complete-source repair proposals as unified diffs; requires explicit approval before applying a patch; reruns approved changes; suggests regression checks; and stores execution and repair history locally.

This paper specifies the current architecture, trust boundaries, data contracts, repair state machine, failure semantics, security controls, and extension points. It also distinguishes properties demonstrated by the implementation from claims that require future empirical evaluation. The architecture is designed around a conservative principle: model output may inform source modification, but it does not itself establish correctness or authority.

Keywords: AI-assisted debugging, human-in-the-loop software repair, sandboxed execution, LLM diagnosis, program repair, verification, developer tooling

1. Design Objectives

The architecture is optimized for six objectives: preserve deterministic evidence; isolate execution from interpretation; constrain model output into a machine-checkable structure; keep source mutation under explicit user control; verify approved changes by execution; and retain enough temporal context to make repair activity auditable. These objectives intentionally favor inspectability over maximal autonomy.

The resulting system is not an autonomous coding agent. It is a debugging workspace in which deterministic components and probabilistic components have different authority. That distinction governs every major module.

2. System Context

Layer	Responsibility	Authority
Interaction	Desktop GUI, CLI, starter VS Code commands	Collect intent; display evidence; receive approval
Analysis	Syntax checks, language routing, project discovery	Deterministic preprocessing
Execution	Docker sandbox or explicit local opt-in	Produce runtime/compiler evidence
Diagnosis	Structured LLM request or offline fallback	Interpret and propose; cannot authorize
Repair	Diff preview, approve/reject, apply	Human-controlled mutation
Verification	Automatic rerun + regression suggestion	Produce post-change evidence
Persistence	History, diffs, parent repair relationships	Record temporal state

3. Reference Data Flow

A failed program traverses a one-way evidence path until a human authorization event permits mutation. The same source may then traverse the execution path again as a verification candidate.

1	Source acquisition	Editor/file/CLI provides source and language.
2	Validation and routing	Language registry selects the appropriate runner; Python may receive live syntax validation.
3	Restricted execution	The program runs in the selected backend and returns success state, output, backend identity, return code, timeout state, and optional trace.
4	Evidence rendering	Runtime output and captured exception state are combined into diagnostic text.
5	Structured diagnosis	AI returns summary, root cause, complete corrected source, explanation, optional regression test, and confidence; provider failure degrades to offline guidance.
6	Patch preview	Corrected source is compared with current source and displayed as a unified diff.
7	Authorization	User explicitly approves or rejects the proposed repair.
8	Mutation	Only approval permits the proposed source to replace the current source.
9	Verification	The repaired source is rerun; outcome is recorded as verified success or failed repair.
10	Persistence	Execution/repair entry stores source, output, success, event type, parent relation, timestamp, and diff from the prior same-language run.

4. Core Data Contracts

The implementation uses compact data structures to prevent loosely formatted model text from becoming an implicit control protocol.

4.1 ExecutionResult

Fields: success, output, backend, return_code, timed_out, trace. The diagnostic_text operation appends captured traceback state to ordinary failure output. This keeps the model-facing evidence derived from an execution object rather than from ad hoc UI strings.

4.2 Diagnosis

Fields: summary, root_cause, corrected_code, explanation, regression_test, confidence, provider. Confidence is clamped to the interval [0,1]. A proposal is considered patch-bearing only when corrected_code is non-empty. The provider field distinguishes AI output from deterministic offline fallback.

4.3 History Entry

History entries contain a UUID, UTC timestamp, event type, optional parent ID, language, complete source, output, success flag, and a unified diff from the previous same-language execution. The parent relationship supports repair lineage rather than merely a flat activity log.

5. Execution Architecture

Execution is language-routed but normalized into a common result. Python, C, C++, and Java use language-specific commands while returning the same ExecutionResult contract. The default backend is Docker; local execution exists as an explicit opt-in path for trusted code.

5.1 Restricted Docker Backend

Control	Current implementation	Purpose
Network	--network none	Prevents ordinary outbound/inbound network access.
Memory	256 MB	Bounds memory consumption.
CPU	0.5 CPU	Bounds processor consumption.
Processes	pids-limit 64	Limits process proliferation.
Filesystem	read-only container root	Reduces writable container surface.
Capabilities	cap-drop ALL	Removes Linux capabilities from the container.
Privilege escalation	no-new-privileges	Blocks acquisition of additional privileges.
Temporary storage	64 MB tmpfs; noexec,nosuid	Provides bounded compiler/runtime scratch space.
Secrets	subprocess environment restricted to PATH	Avoids inheriting the host API key into executed code.
Workspace	temporary bind mount	Exposes only the temporary program workspace.

These controls constitute defense in depth, not a proof of perfect isolation. The project explicitly avoids representing Docker as an operating-system-grade security boundary and does not position the prototype as a hosted untrusted-code execution service.

5.2 Python Exception-State Capture

For Python failures, a trace harness walks the exception traceback and captures frames belonging to the user program. Each frame records file, line, function, and local variables. Local representations are bounded to 300 characters and representation failures degrade to an explicit placeholder. Trace metadata is emitted through a dedicated marker on stderr, removed from the user-visible error stream, parsed into structured JSON, and attached to ExecutionResult.

This mechanism improves diagnostic context without claiming full program tracing. Successful statements are not comprehensively traced.

5.3 Compilation Paths

C and C++ compile into the container's temporary filesystem before execution. Java compiles classes into temporary storage and runs them from that location. Compilation and runtime behavior therefore share the same resource and network restrictions as interpreted execution.

6. AI Diagnosis Boundary

The model receives source code and error evidence and is instructed to return one JSON object with a fixed schema. Required fields are validated before a Diagnosis object is constructed. The prompt explicitly prohibits claiming that a correction has been verified and asks the model to preserve user intent while avoiding unrelated rewrites.

Provider failures, missing source, missing error output, or an absent API key degrade to offline diagnosis. Offline mode returns deterministic troubleshooting guidance and no corrected source. Thus model availability is an enhancement rather than a prerequisite for basic debugging.

7. Human-Governed Repair Protocol

The repair protocol is the central control structure of the current architecture. It converts model output into a candidate state that must pass a human gate before becoming executable source.

State	Permitted transition	Invariant
FAILED	DIAGNOSED	Failure evidence exists before AI interpretation.
DIAGNOSED	PREVIEWED or NO_PATCH	Diagnosis is structured; model confidence is advisory.
PREVIEWED	APPROVED or REJECTED	Candidate mutation is visible before authorization.
REJECTED	FAILED / editing	Source is not mutated by the rejected proposal.
APPROVED	APPLIED	Mutation is attributable to explicit user action.
APPLIED	VERIFYING	Applied code must be rerun before a success claim.
VERIFYING	VERIFIED or REPAIR_FAILED	Verification outcome is determined by execution evidence.

7.1 Why Complete Replacement Source?

The diagnosis contract asks for complete replacement source rather than an unstructured prose patch. The application can therefore compute its own unified diff against the current source. This separates the model's proposed target state from the application's presentation of the change.

7.2 Why Approval Is Not Verification

Approval answers an authorization question: may this change be applied? Verification answers an empirical question: what happened when the approved code ran? Conflating the two would allow user trust or model confidence to masquerade as execution evidence. The architecture keeps them separate.

7.3 Regression Suggestions

The model may supply a regression test or minimal reproducible check. This is displayed as a suggested safeguard, not as proof that a comprehensive test suite has passed. A future evaluation layer can execute and score such tests more systematically.

8. Project Context and Context Governance

Project discovery recursively selects supported source files while ignoring common dependency, environment, cache, and version-control directories. Collection stops at either 40 files or 250,000 bytes by default. Supported extensions are .py, .c, .cpp, and .java.

Bounding context serves four architectural purposes: relevance control, privacy reduction, predictable model cost, and resistance to accidental prompt inflation. The current implementation uses simple deterministic limits rather than learned retrieval. That simplicity makes the context boundary inspectable, although it may omit semantically important files in larger projects.

9. Persistence and Temporal Debugging

ExecutionHistory persists locally to a JSON file under the user's home directory. Writes use a temporary file followed by replacement, reducing the chance of partially written history. Each new entry computes a unified diff against the latest run in the same language. Repair events can point to a parent execution, allowing the system to preserve causal lineage between a failure and its attempted correction.

10. Interface and Integration Architecture

The desktop workspace is a Tkinter IDE-style interface with a line-numbered editor, resizable history and result regions, and separate output, diagnosis, diff, and regression-test surfaces. Run, diagnose, and apply/verify operations are exposed as distinct user actions. Background AI requests preserve UI responsiveness.

The CLI exposes execution, diagnosis, and project inspection for scripting and reproducible terminal workflows. Starter VS Code commands provide an integration seam without making the core engine dependent on a specific editor. This separation permits future front ends to reuse the same execution and diagnosis contracts.

11. Trust Boundaries and Threat Model

Boundary	Primary risk	Current mitigation
Host -> executed code	Malicious or runaway program	Docker default; network/resource/capability/filesystem restrictions; local path requires opt-in.
Host secrets -> executed code	Credential exposure	Execution subprocess receives restricted environment rather than inherited API secrets.
Program evidence -> model	Oversharing or prompt inflation	Bounded project discovery and bounded local representations.
Model -> source tree	Incorrect/destructive repair	Structured proposal, diff preview, explicit approval gate.
Approved source -> correctness claim	Plausible but broken repair	Automatic rerun; outcome recorded as verified or failed.
Persistence -> interpretation	History mistaken for learning	History records state; it does not autonomously retrain or alter the model.

The threat model is intentionally conservative. Docker configuration reduces risk but does not justify claims of absolute containment. Likewise, approval gating reduces unintended mutation but does not guarantee that a user-approved repair is semantically correct.

12. Failure Semantics

Failure states are treated as first-class outputs rather than exceptional control-flow accidents. Unsupported languages return a normalized failure. Unknown backends return a failure. Missing Docker returns an unavailable state with explicit opt-in instructions for trusted local execution. Timeouts are marked separately. AI-provider exceptions degrade to offline diagnosis. Corrupt history files reset in-memory history rather than preventing application startup.

This degradation strategy preserves a usable debugging core even when optional infrastructure fails.

13. Demonstrated Properties vs. Open Research Claims

Demonstrated by implementation	Requires dedicated evaluation
Four-language routing and execution paths	Comparative repair accuracy across languages
Structured diagnosis parsing	Calibration quality of model confidence
Explicit approve/reject repair gate	Human factors: whether the gate improves decision quality
Automatic rerun after approved mutation	Statistical reduction in time-to-fix
Restricted Docker command configuration	Adversarial sandbox escape resistance
Bounded Python exception-state capture	Value of local-state context versus traceback-only prompts

Demonstrated by implementation	Requires dedicated evaluation
Persistent history and unified diffs	Longitudinal effect on developer productivity
Deterministic offline demo generation	Generalization to large real-world repositories

14. Design Tradeoffs

Full-source replacement vs. minimal patch generation. Full replacement simplifies the model contract and lets the application derive the diff, but it can produce unnecessarily broad edits. Future versions can introduce validated patch formats while retaining the preview/approval invariant.

Simple bounded project discovery vs. semantic retrieval. Deterministic file and byte limits are transparent and cheap, but they are not relevance-aware. Retrieval can improve context quality later, provided it remains auditable and bounded.

Docker portability vs. stronger isolation. Docker is widely available and practical for a desktop prototype. Stronger isolation mechanisms could improve adversarial resistance at the cost of platform complexity.

Human approval vs. autonomous throughput. Approval adds friction. That friction is intentional at the current maturity level because the cost of an incorrect silent mutation is greater than the cost of one explicit decision.

15. Extension Points

The architecture admits several extensions without discarding its trust model: additional language runners behind the common execution contract; semantic project retrieval under explicit context budgets; executable regression-test orchestration; static-analysis and type-checker evidence; patch schemas with line-level preconditions; policy-based approval for low-risk changes; multiple diagnostic agents with role separation; and benchmark instrumentation for repair quality, latency, and human interaction.

A future multi-agent debugger should preserve the current authority hierarchy. Agents may debate diagnosis or generate competing repairs, but execution evidence remains deterministic, source mutation remains policy-governed, and verification remains separate from proposal generation.

16. Reference Invariants

ID	Invariant
I1	No AI response is treated as verified merely because it is well-formed or high-confidence.
I2	A repair proposal does not mutate source before an explicit authorization event.
I3	An approved repair is rerun before being recorded as verified.
I4	Executed code does not receive the host OpenAI API key through inherited environment state.
I5	Local execution is not the implicit default.
I6	Project context and captured local representations are bounded.
I7	Offline/provider failure does not disable deterministic execution.
I8	Persistent history records events; it is not represented as autonomous learning.

17. Limitations

The current implementation does not provide operating-system-grade isolation without Docker, trace every successful statement, or execute arbitrary multi-service projects as a single unit. Project context selection is size-bounded rather than semantically optimized. The diagnosis path currently depends on a single model request when AI is enabled. Regression tests are suggested rather than universally executed. Persistent history is local JSON rather than a transactional event store. These constraints define the prototype's present engineering boundary.

18. Conclusion

AI Debugger Pro's current architecture is built around a narrow but consequential proposition: useful AI autonomy does not require surrendering control of mutation or correctness claims. By separating deterministic execution, structured interpretation, visible proposal, human authorization, and post-change verification, the system creates a debugging loop in which each component has explicit authority and explicit limits.

This architecture is suitable as the reference baseline for the project's next stage. Future work can increase intelligence, context depth, automation, and agent count without weakening the central protocol. The path forward is not to remove the gates blindly; it is to gather enough evidence to know which gates can safely become policies, which must remain human decisions, and which should never be delegated.

Appendix A - Module Responsibility Map

Module	Responsibility
core/analyzer.py	Python syntax validation.
core/executor.py	Local language execution and compiler/runtime output capture.
core/sandbox.py	Restricted Docker execution; normalized ExecutionResult; Python exception-state capture.
core/project.py	Bounded multi-file source discovery and rendered project context.
core/languages.py	Language registry and routing.
core/ai_suggester.py	Structured AI diagnosis request, schema parsing, and offline fallback.
core/diagnostics.py	Immutable Diagnosis contract and confidence normalization.
core/history.py	Persistent execution/repair events, parent relationships, and same-language diffs.
core/diff.py	Unified source-code diff generation.
interface/gui.py	Desktop interaction, preview, approval, repair, and result presentation.
cli.py	Scriptable run, diagnose, and project-inspection commands.
vscode-extension/	Starter editor command integration.

Appendix B - Publication and Evaluation Roadmap

For a future peer-reviewed submission, this reference architecture should be paired with an empirical evaluation paper rather than overloaded with unsupported performance claims. Recommended experiments include: a stratified bug corpus across supported languages; baseline comparisons against traceback/compiler-only debugging and one or more AI coding assistants; blinded scoring of root-cause accuracy and patch correctness; pass@1 repair rate; regression-introduction rate; time-to-diagnosis and time-to-verified-repair; ablation of exception locals and project context; human approval acceptance/rejection accuracy; confidence calibration; and adversarial tests of the execution boundary.

Reporting should include model/provider/version, prompt protocol, hardware/runtime environment, compiler/JDK versions, Docker version, timeout/resource policy, dataset construction, exclusion criteria, and raw aggregate counts. The project should avoid claims such as 'safer,' 'faster,' or 'more accurate' until those terms are operationalized and measured.

References

1. Bentley, T. R. *AI Debugger Pro: A Multi-Language Debugging Framework with LLM-Assisted Error Interpretation and Execution Tracing*. Project paper, 2026.
2. Bentley, T. R. *From Stateless Execution to Human-Governed AI Repair: The Evolution of AI Debugger Pro, Versions 0-6*. System evolution case study, 2026.
3. Tybent18/ai-debugger-pro. Current master-branch source and README, accessed September 2026.

Document role: authoritative architecture reference for the current prototype. Performance, comparative effectiveness, and human-factors claims are intentionally reserved for empirical evaluation.